Práctica Sincronización

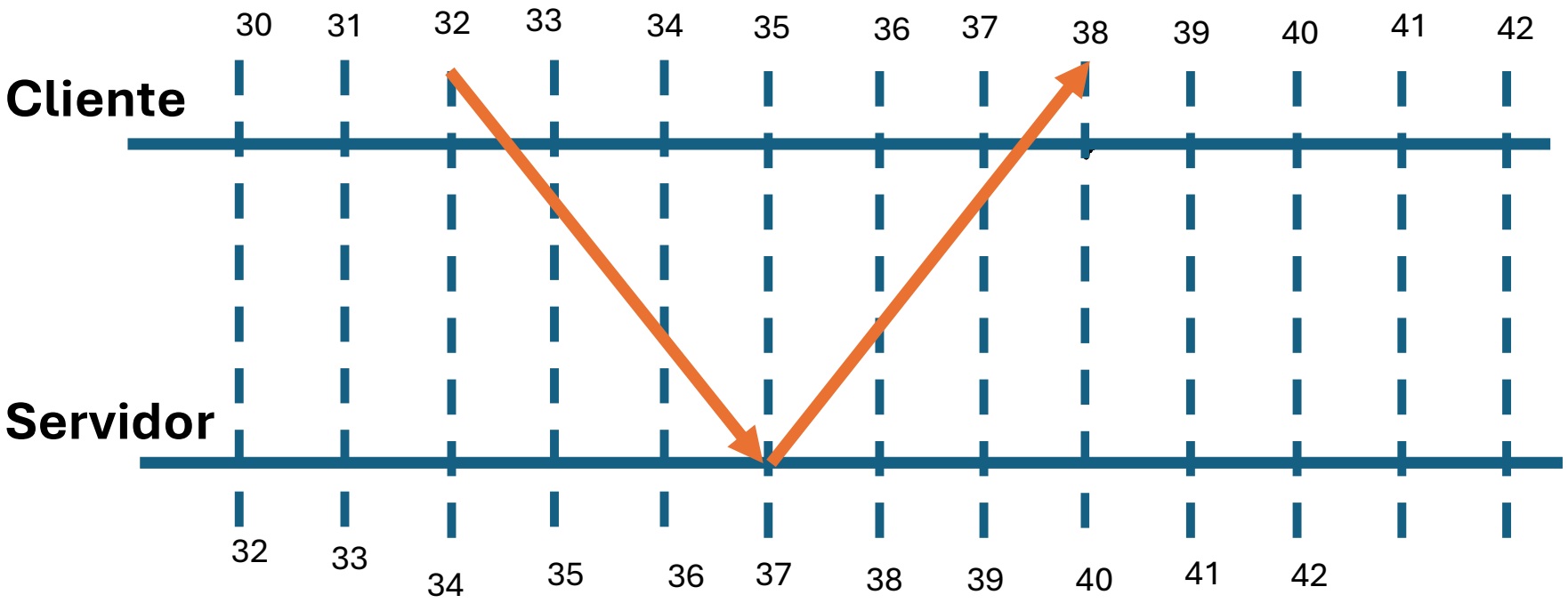
Sistemas Distribuidos

Profesora. Elba Karen Sáenz García

Cristian

El servidor contesta con el valor de su reloj Cs(37)

La respuesta llega al cliente en T1 (según Cc en 38)



$$C = Cs + (T1 - T0)/2$$

Servidor

```python
import socket
import time

HOST = "0.0.0.0"
PORT = 5000

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((HOST, PORT))
server.listen()

print("Servidor de tiempo activo...")

while True:
    conn, addr = server.accept()
    current_time = time.time()
    conn.send(str(current_time).encode())
    conn.close()
```

Server

```
import socket
import time
HOST = "0.0.0.0"
PORT = 5000
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((HOST, PORT))
server.listen()
print("Servidor de tiempo activo...")
while True:
    conn, addr = server.accept()
    Cs = time.time()
    conn.send(str(Cs).encode())
    conn.close()
```

Cliente

```
import socket
import time
HOST = "127.0.0.1"
PORT = 5000
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM) # IPv4, TCP
client.connect((HOST, PORT))
t0 = time.time()
tiempoServi = float(client.recv(1024).decode())
t1 = time.time()
client.close() # Cierra la conexión del socket.
RTT = t1 - t0
C = tiempoServi+ RTT/2 #tiempo estimado
print("Tiempo servidor:", tiempoServi)
print(f"T1-t0:{RTT},Tiempo Cc {t1}, Tiempo ajustado a C={C} ")
```

Actividad 1

Simular latencia en red agregando

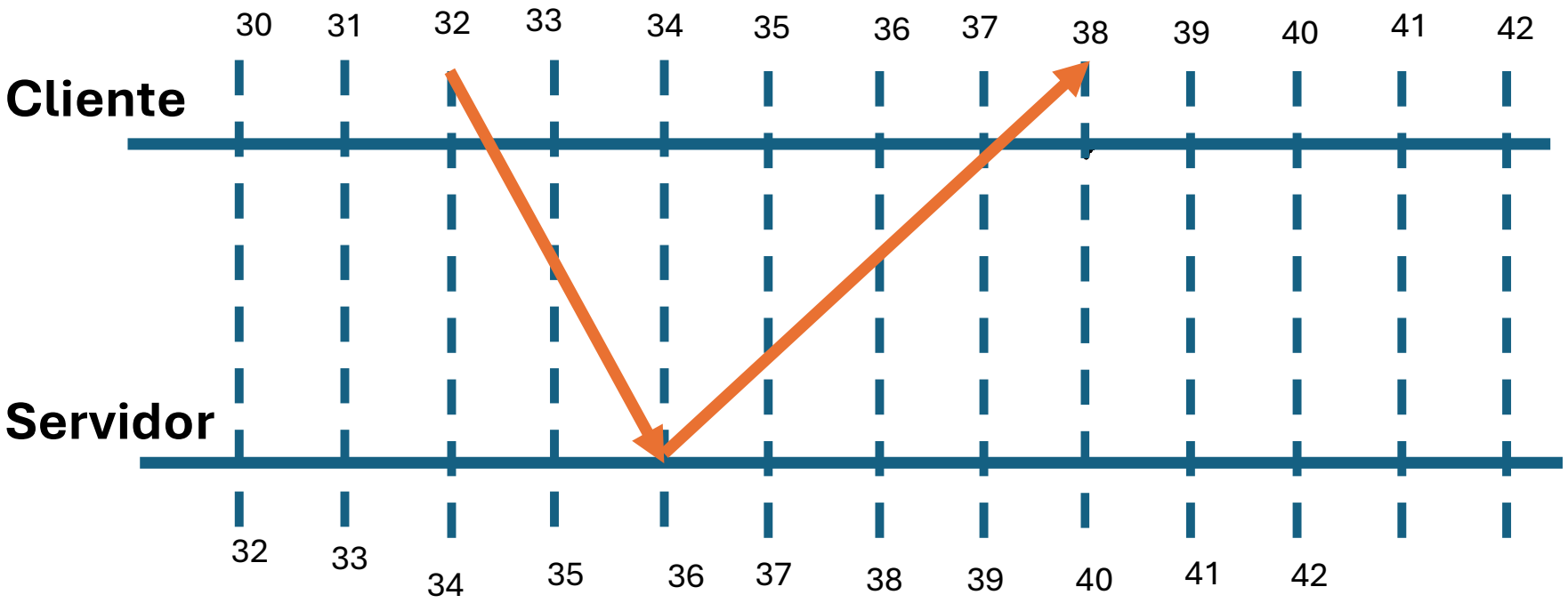
```
 delay = 2  # segundos
time.sleep(delay)
```

¿cómo cambia el ajuste del reloj?

Cristian

El servidor contesta con el valor de su reloj Cs(37)

La respuesta llega al cliente en T1 (según Cc en 38)



$$C = Cs + (T1 - T0)/2$$

Actividad 2

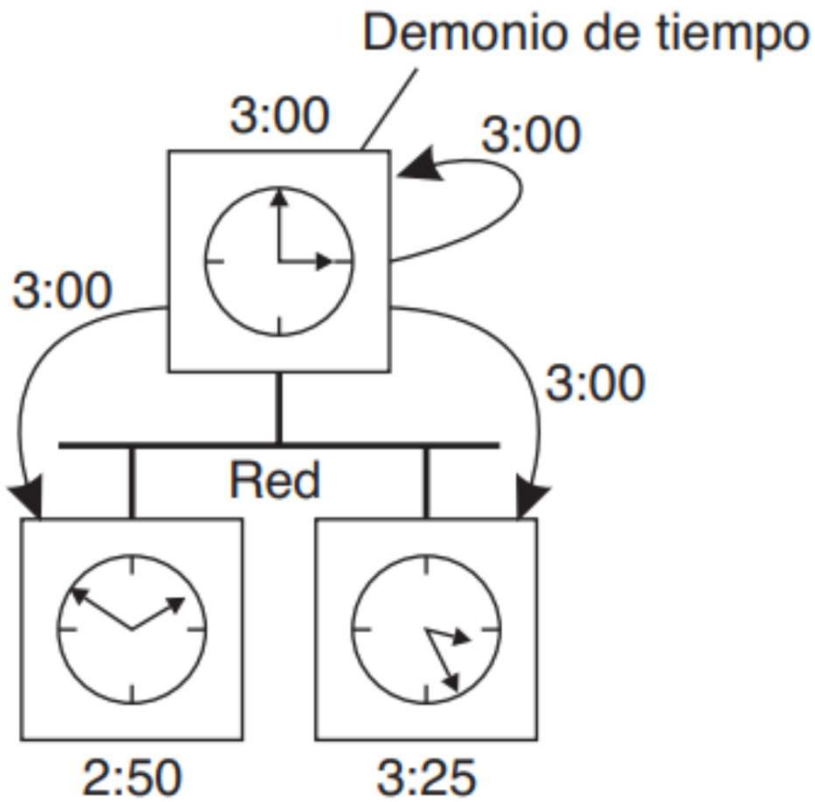
- Modificar la implementación anterior para que se pueda trabajar con varios clientes al mismo tiempo (máquinas diferentes, pueden ser físicas o virtuales y multithread) y se vea una diferencia de tiempo.
- ¿todos obtienen el mismo ajuste?¿por qué?

- Agregar al código lo necesario para considerar cuando C > Cc o C<Cc.

Berkeley

Berkeley

- a las 3:00, el demonio de tiempo indica a las otras máquinas su hora y les pregunta la hora que tienen

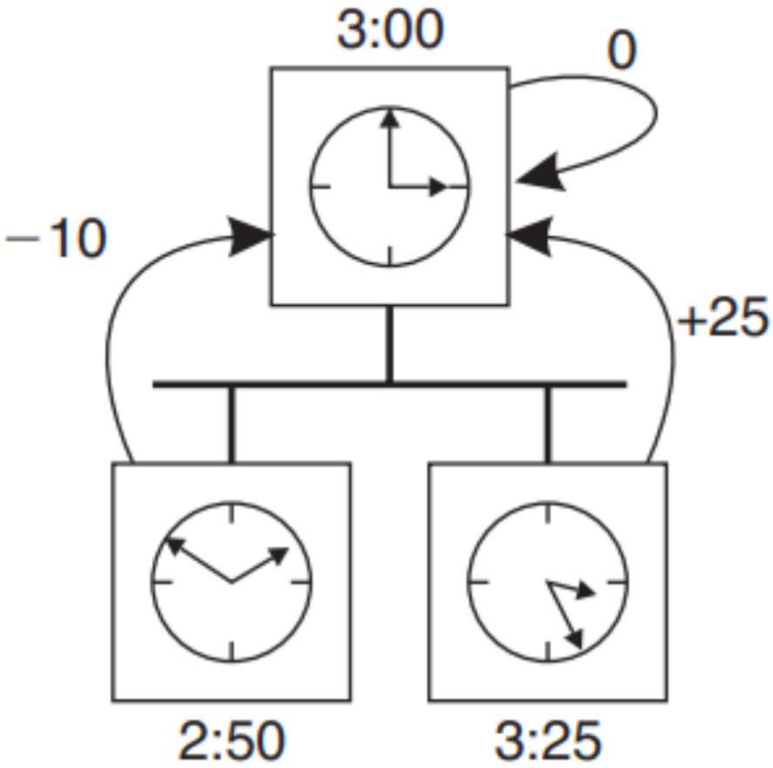


(a)

Berkeley

- las máquinas responden qué tanto se encuentran adelante o detrás del demonio de tiempo.

- (-10+25+0)/3=5



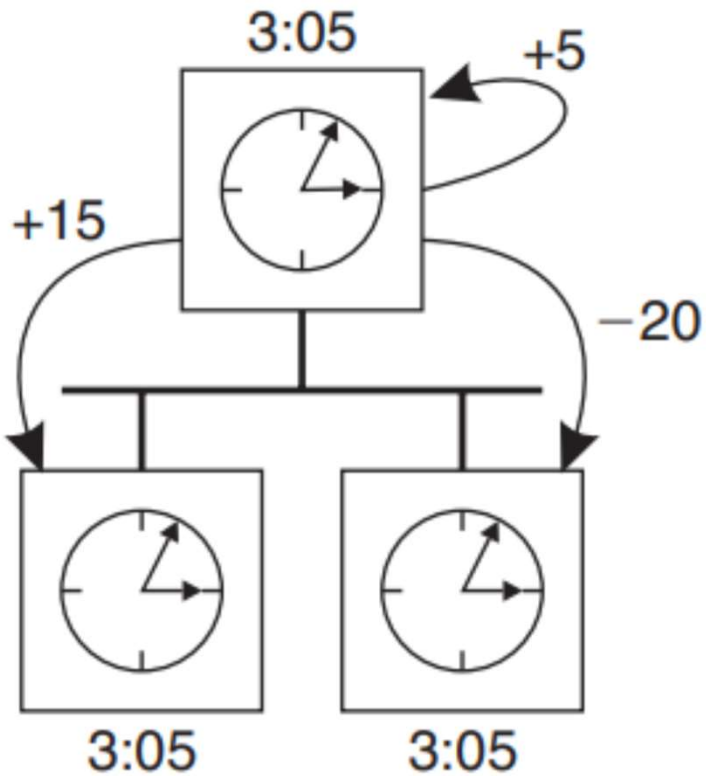
(b)

Berkeley

- Con estos números, el demonio calcula el promedio e indica a cada máquina cómo ajustar su reloj.
- El tiempo sincronizado será: (-10+25+0)/3=5

Cada nodo se ajusta para llegar a 3:05

- 3:00 +5=3:05
- 2:50 +15=3:05
- 3:25-20=3:05



(c)

Funcionamiento

Nodo	Hora
Servidor	10:00:00
Cliente A	10:00:06
Cliente B	09:59:58
Cliente C	10:00:04

¿Todos están sincronizados?

Nodo	Hora	Diferencia enviada al servidor por cada máquina
Servidor	10:00:00	0 s
A	10:00:06	+6 s
B	09:59:58	−2 s
C	10:00:04	+4 s

Cálculo de promedio y tiempo global por parte del servidor

Servidor calcula promedio	Y nuevo tiempo global
• (0 + 6 - 2 + 4)/4 = 8/4=2	• 10:00:00 + 2 s = 10:00:02

Envio de ajuste a clientes

Nodo	Hora actual	Ajuste	Nueva hora
Servidor	10:00:00	+2 s	10:00:02
A	10:00:06	−4 s	10:00:02
B	09:59:58	+4 s	10:00:02
C	10:00:04	−2 s	10:00:02

```python
import socket
import struct
import time
import random
from multiprocessing import Process
```

```python
def servidorBerkeley(port=60001, num_clients=3):
    sockServer = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sockServer.bind(("localhost", port))
    sockServer.listen(num_clients)
    print("Servidor maestro esperando a los clientes...")
    connections = []
    times = []
    # Aceptar conexiones
    for _ in range(num_clients):
        conn, addr = sockServer.accept()
        print(f"Conexión establecida con {addr}")
        print("")
        connections.append(conn)
```

```python
# Solicitar tiempos
for conn in connections:
    conn.send(b'TIME_REQUEST')
# Recibir tiempos
for conn in connections:
    client_time = struct.unpack('f', conn.recv(4))[0]
    print(f"Tiempo recibido del cliente: {client_time}")
    times.append(client_time)
```

```python
# Tiempo del servidor
server_time = time.time()
print(f"Tiempo del servidor (maestro): {server_time}")
print("")
# calcula  PROMEDIO
all_times = times + [server_time]
average_time = sum(all_times) / len(all_times)

print(f"Tiempo promedio calculado por el servidor: {average_time}")
print("")
# Calcular ajustes para clientes
adjustments = [average_time - client_time for client_time in times]

# Ajuste del servidor
server_adjustment = average_time - server_time
print(f"Ajuste del servidor: {server_adjustment}")
```

```python
# Enviar ajustes
for conn, adjustment in zip(connections, adjustments):
    print(f"Enviando ajuste de {adjustment} al cliente")
    print("")
    conn.send(struct.pack('f', adjustment))

# Cerrar conexiones
for conn in connections:
    conn.close()

sockServer.close()
```

```python
def clienteBerkeley(port=60001):
    sockClient = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sockClient.connect(("localhost", port))
    # Simular reloj con desfase
    local_time = time.time() + random.uniform(-5, 5)
    print(f"Tiempo local del cliente antes del ajuste: {local_time}")

    # Esperar solicitud
    request = sockClient.recv(1024)
    if request == b'TIME_REQUEST':
        sockClient.send(struct.pack('f', local_time))

    # Recibir ajuste
    adjustment = struct.unpack('f', sockClient.recv(4))[0]
    print(f"Ajuste recibido del servidor: {adjustment}")
    # Aplicar ajuste
    new_time = local_time + adjustment
    print(f"Nuevo tiempo ajustado por cliente: {new_time}")

    sockClient.close()
```

```python
def ejecutaClients(num_clients=3):
    processes = []

    for _ in range(num_clients):
        p = Process(target=clienteBerkeley)
        processes.append(p)
        p.start()
        time.sleep(1)

    for p in processes:
        p.join()
```

```python
if __name__ == "__main__":
    server_process = Process(target=servidorBerkeley)
    server_process.start()

    time.sleep(2)

    ejecutaClients(num_clients=3)

    server_process.join()
```

Notas

- Cada cliente inicia con un reloj distinto
- El servidor calcula un promedio
- Todos convergen a un mismo tiempo
- Ejecutar el programa y responder:
  - ¿Cuál era el tiempo inicial de cada cliente?
  - ¿Cuál fue el promedio?
  - ¿A qué tiempo convergen?

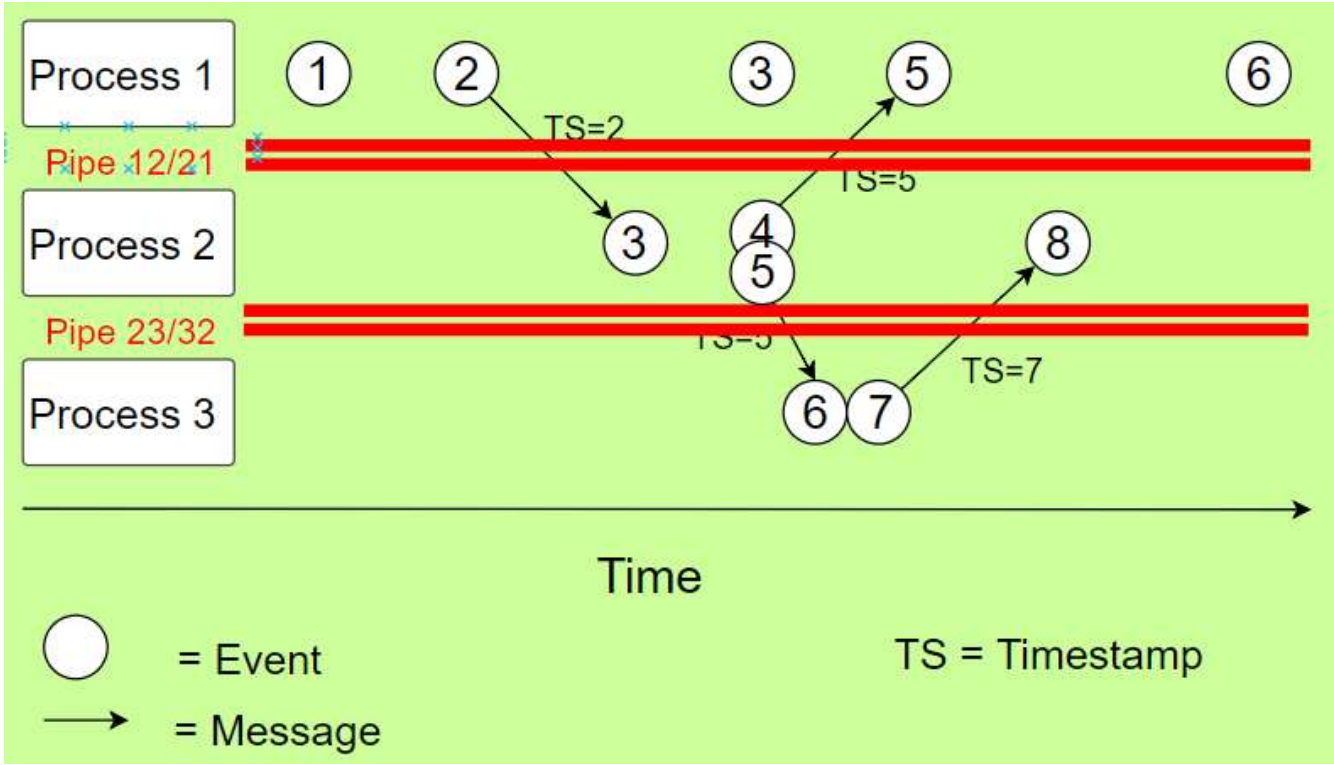
Actividades

- **Actividad 3.** Modificar el programa para trabajar con:
  - 5, 10 y 20 clientes
  - Respondera las preguntas:
    - ¿Cómo cambia el tiempo promedio?
    - ¿El sistema sigue siendo estable?
- **Actividad 4: Simulación de latencia de red**
  - Agrega un retraso aleatorio en los clientes:
    - time.sleep(random.uniform(0, 3))
  - Responder a las preguntas:
    - ¿La latencia afecta el resultado final?
    - ¿Por qué el algoritmo de Berkeley es sensible o no a este factor?
- **Actividad 5 :** Modificar la implementación anterior para ejecutar, servidor y clientes en máquinas diferentes.

Lamport

- **Actividad 6**. Para la implementación del algoritmo de relojes lógicos de Lamport se proporciona un programa en Python que utiliza el módulo *multiprocessing* para la generación de procesos locales.
- El programa es de acuerdo a un ambiente de eventos establecido de cada proceso. Figura 1.
- Se pide probarlo y entenderlo para poder plantear en equipo un ambiente sugerido con 7 procesos.
- Entregar diagrama de ambiente propuesto

Fig1


Process 1
Pipe 12/21
Process 2
Pipe 23/32
Process 3
1
2
3
5
6
TS=2
TS=5
3
4
5
8
TS=7
6
7
Time
= Event
= Message
TS = Timestamp

```python
from multiprocessing import Process, Pipe
from os import getpid
from datetime import datetime


def local_time(counter):
    return ' (LAMPORT_TIME={}, LOCAL_TIME={})'.format(counter, datetime.now()
)
#calcula la     nueva marca de tiempo
def calc_recv_timestamp(recv_time_stamp, counter):
    return max(recv_time_stamp, counter) + 1

#función para cada evento que pueda ocurrir
def event(pid, counter):
    counter += 1
    print('Algo sucedió en el proceso {}!'.format(pid) + local_time(counter))
    return counter
```

```python
def send_message(pipe, pid, counter):
    counter += 1
    pipe.send(('Mensaje', counter))
    print('Mensaje enviado desde el proceso {}!'.format(pid) + local_time(counter))
    return counter

def recv_message(pipe, pid, counter):
    message, timestamp = pipe.recv()
    counter = calc_recv_timestamp(timestamp, counter)
    print('Mensaje enviado a  ' + str(pid) + local_time(counter))
    return counter
```

```python
def process_one(pipe12):
    pid = getpid()
    counter = 0
    counter = event(pid, counter)
    counter = send_message(pipe12, pid, counter)
    counter  = event(pid, counter)
    counter = recv_message(pipe12, pid, counter)
    counter  = event(pid, counter)

def process_two(pipe21, pipe23):
    pid = getpid()
    counter = 0
    counter = recv_message(pipe21, pid, counter)
    counter = send_message(pipe21, pid, counter)
    counter = send_message(pipe23, pid, counter)
    counter = recv_message(pipe23, pid, counter)
```

```python
def process_three(pipe32):
    pid = getpid()
    counter = 0
    counter = recv_message(pipe32, pid, counter)
    counter = send_message(pipe32, pid, counter)

if __name__ == '__main__':
    oneandtwo, twoandone = Pipe()
    twoandthree, threeandtwo = Pipe()
    #creacion de procesos
    process1 = Process(target=process_one,args=(oneandtwo,))
    process2 = Process(target=process_two,args=(twoandone, twoandthree))
    process3 = Process(target=process_three,args=(threeandtwo,))
    process1.start()
    process2.start()
    process3.start()
    process1.join()
    process2.join()
    process3.join()
```

Actividad 7.

- Con la idea anterior implementar una solución al algoritmo de relojes vectoriales, plantear un ambiente con 5 procesos. Entregar la figura con el ambiente sugerido que se programará.